

INTRODUCTION TO SOFTWARE ENGINEERING AND SOFTWARE PROCESS

1. SOFTWARE AND SOFTWARE ENGINEERING

1.1. *The changing Nature of Software*

Software has a dual role. Software is a product and also the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware or, more broadly, a network of computers that are accessible by local hardware. Sometimes software resides within a cellular phone or operates inside a mainframe computer. Software is an information transformer—producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as a multimedia presentation. As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).

Software delivers the most important product of our time—information. Some of the functions of Software are:-

1. It transforms personal data so that the data can be more useful in a local context;
2. It manages business information to enhance competitiveness;
3. It provides a gateway to worldwide information networks and provides the means for acquiring information in all of its forms.

The role of computer software has undergone significant change over the last half-century. Dramatic improvements in hardware performance, profound changes in computing architectures, vast increases in memory and storage capacity, and a wide variety of exotic input and output options have all precipitated more sophisticated and complex computer-based systems. Sophistication and complexity can produce dazzling results when a system succeeds, but they can also pose huge problems for those who must build complex systems.

Today, a huge software industry has become a dominant factor in the economics of the industrialized world. Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era. And yet, the same questions asked of the lone programmer are being asked when modern computer-based systems are built:

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all the errors before we give the software to customers?
- Why do we continue to have difficulty in measuring progress as software is being developed?

These, and many other questions, are a manifestation of the concern about software and the manner in which it is developed—a concern that has led to the adoption of software engineering practice.

1.2. *Defining A Software*

Software is a comprehensive term used to describe a collection of computer programs, procedures, and related data that provide the instructions for telling a computer what to do and how to do it. Unlike hardware, software is intangible — it cannot be touched or physically handled.

Software does not wear out over time, but it does deteriorate in terms of performance due to updates, changes, and external interactions. As systems evolve, software needs constant maintenance and enhancements to remain effective.

Key Characteristics of Software:

- a. **Intangibility:** Software cannot be physically touched like hardware components.
- b. **Engineered, Not Manufactured:** Software is developed or engineered using specific methodologies and tools. It does not go through a manufacturing process like physical products. Although some similarities exist between software development and hardware manufacture, the two activities are fundamentally different. In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software. Both activities are dependent on people, but the relationship between people applied and work accomplished is entirely different. Both activities require the construction of a "product" but the approaches are different. Software costs are concentrated in engineering. This means that software projects cannot be managed as if they were manufacturing projects.
- c. **Software doesn't "wear out.":** Figure 1.1 depicts failure rate as a function of time for hardware. The relationship, often called the "bathtub curve" indicates that hardware exhibits relatively high failure rates early in its life; defects are corrected and the failure rate drops to a steady-state level (ideally, quite low) for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the **hardware begins to wear out**.
Software is not susceptible to the environmental maladies that cause hardware to wear out. In theory, therefore, the failure rate curve for software should take the form of the "idealized curve" shown in Figure 1.2. Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected (ideally, without introducing other errors) and the curve flattens as shown. However, the implication is clear—software doesn't wear out. But it does deteriorate!

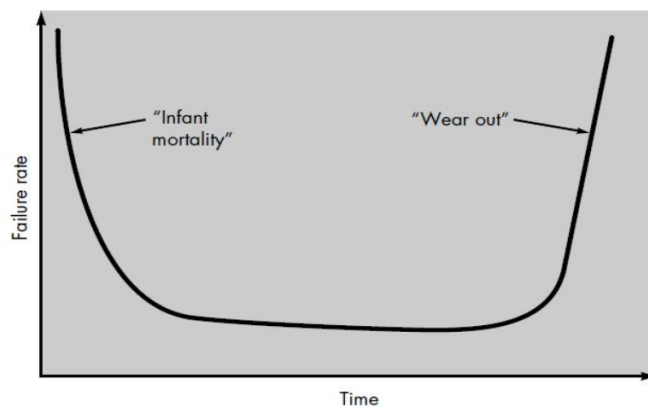


Figure 1.1 Failure curve for hardware

This seeming contradiction can best be explained by considering the “actual curve” shown in Figure 1.2. During its life, software will undergo change (maintenance). As changes are made, it is likely that some new defects will be introduced, causing the failure rate curve to spike as shown in Figure 1.2. Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise—the software is deteriorating due to change. Another aspect of wear illustrates the difference between hardware and software. When a hardware component wears out, it is replaced by a spare part. There are no software spare parts. Every software failure indicates an error in design or in the process through which design was translated into machine executable code. Therefore, software maintenance involves considerably more complexity than hardware maintenance.

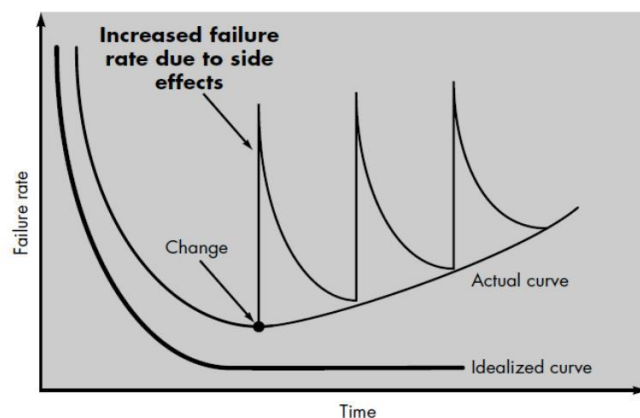
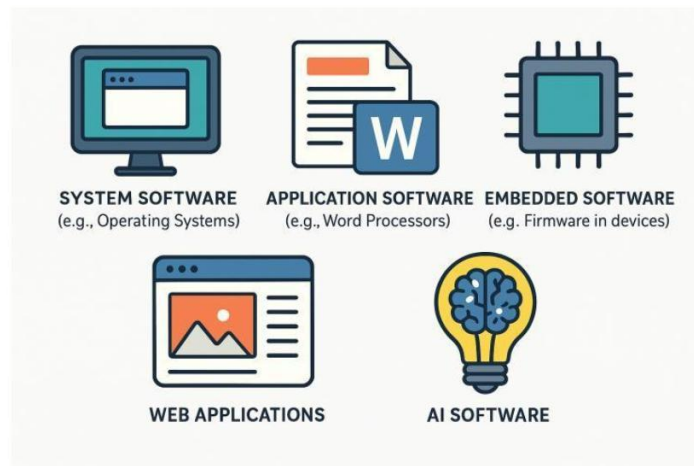


Figure 1.2 Failure curves for Software

- d. Maintenance:** The majority of software development effort goes into maintenance after the initial development to fix bugs, adapt to new platforms, or enhance performance.
- e. Complexity:** Software systems are often incredibly complex, with thousands or even millions of lines of code that interact in intricate ways.

Types of Software:



- ▮ System Software (e.g., Operating Systems)
- ▮ Application Software (e.g., Word Processors)
- ▮ Embedded Software (e.g., Firmware in devices)
- ▮ Web Applications
- ▮ AI Software

1) System Software

System software is designed to operate, manage, and support the computer hardware and system resources. It provides a platform for running application software.

Why it is classified as System Software:

- ▮ It runs in the background and manages system-level functions.
- ▮ Essential for the basic functioning of a computer.
- ▮ Directly interacts with hardware.

Examples:

- ▮ **Operating Systems** (e.g., Windows, Linux, and mac OS): Manage hardware and provide a user interface.
- ▮ **Device Drivers** (e.g., printer drivers): Help hardware communicate with the OS.
- ▮ **Utilities** (e.g., disk clean up tools, antivirus): Perform maintenance tasks.

2) Application Software

Application software is designed to help users perform specific tasks or solve particular problems.

Why it is classified as Application Software:

- ▮ It runs on top of system software.
- ▮ Created for end-users to carry out productivity tasks.

Examples:

- ▮ **MS Word, Excel:** Used for document editing and data analysis.
- ▮ **Media Players (VLC):** Play audio and video files.
- ▮ **Browsers (Chrome, Firefox):** Access web content.

3) Embedded Software

Embedded software is specialized software designed to run on embedded systems (hardware with limited computing resources, often dedicated to specific tasks).

Why it is classified as Embedded Software?

- ▮ Runs on non-general-purpose computers.
- ▮ Controls hardware-specific functions.
- ▮ Stored in firmware or ROM.

Examples:

- ▮ **Software in Washing Machines:** Controls washing cycles.
- ▮ **Automobile ECU (Engine Control Unit):** Manages engine performance.
- ▮ **Smart TVs, Microwaves, Smart watches:** Software controls device behaviour.

4) Web-based and AI Software

a. Web-Based Software

Software that is hosted on web servers and accessed via web browsers over the internet.

Why it is classified as Web-Based Software:

- ▮ Operates online without local installation.
- ▮ Depends on client-server architecture.

Examples:

- ▮ **Google Docs:** Online document editing.
- ▮ **YouTube:** Video streaming service.
- ▮ **Online Banking Portals**

b. AI Software

Software that exhibits intelligent behaviour and mimics human cognitive functions like learning, problem-solving, and decision-making.

Why it is classified as AI Software:

- ▮ Implements algorithms that allow machines to learn and adapt.
- ▮ Based on artificial intelligence technologies like machine learning, NLP, and computer vision.

Examples:

- ▮ **Chatbots (e.g., ChatGPT):** Natural language interaction.
- ▮ **Voice Assistants (Alexa, Siri):** Understand and respond to voice commands.
- ▮ **Face Recognition Systems**

5) Business Software

Business software refers to applications developed specifically for business functions like accounting, HR, project management, and CRM.

Why it is classified as Business Software:

- ▮ Supports commercial operations and improves efficiency.
- ▮ Tailored to enterprise needs.

Examples:

- ▮ **Tally ERP:** Accounting software.
- ▮ **Salesforce:** Customer Relationship Management (CRM).
- ▮ **SAP, Oracle ERP:** Enterprise resource planning.

Software Type	Purpose	Examples
System Software	Manages hardware and system resources	Windows, Linux, Device Drivers
Application Software	Helps users perform specific tasks	MS Office, VLC, Chrome
Embedded Software	Controls specific hardware devices	Washing Machine Software, Smart TV OS
Web-based Software	Accessible via browsers over the internet	Google Docs, YouTube
AI Software	Simulates human intelligence	ChatGPT, Face Recognition
Business Software	Used for commercial and business operations	Tally, SAP, Salesforce

1.1. What is Software Engineering?

Software Engineering can be defined as:

1. ***"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software".*** - IEEE Standard Glossary of Software Engineering Terminology
2. ***"Software engineering is a layered technology and a disciplined approach for software development that includes methods, tools, and procedures for building software systems."***
— **Roger S. Pressman & Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 9th Edition**
3. **"Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use."**
— *Ian Sommerville, Software Engineering*

In simple form software Engineering can be defined as "**Software engineering is the process of designing, creating, testing, and maintaining software applications by applying engineering principles**".

Software Engineering integrates the principles of engineering with the requirements of software development. It aims to produce high-quality software that is reliable, maintainable, and efficient while being delivered within budget and time constraints.

Core Objectives of Software Engineering:

- a. Deliver software that meets user needs and expectations.
- b. Maintainability: Ability to modify software easily when requirements change.
- c. Efficiency and performance.
- d. Scalability and portability across different platforms.
- e. Effective project management to avoid cost and time overruns.

Software Engineering Practices include:

- a. Understanding the problem thoroughly
- b. Planning before designing
- c. Clear and organized design process
- d. Regular testing and debugging
- e. Continuous improvement of the development process

1.2. The Software Process

The software process is a framework that includes the structured set of activities required to build software systems. It consists of:

- a. **Communication:** This phase involves gathering requirements and understanding the needs of the stakeholders through interaction and feedback.
- b. **Planning:** Estimation of costs, resources, timelines, and scheduling tasks for development. Risk management is also considered here
- c. **Modeling:** Creating abstract representations (models) of the software, including architecture, component design, and interface specifications.
- d. **Construction:** Coding the software based on design models. It includes unit testing and integration testing.
- e. **Deployment:** Delivery of the software to the user and subsequent maintenance based on feedback and error reports.

1.3. Software Engineering Practices

Modern software engineering practice includes a combination of methodologies, tools, and workflows to deliver effective software solutions.

Best Practices Include:

- a. **Requirement Analysis:** Understanding customer needs through techniques like interviews, use cases, and prototyping.
- b. **Software Design:** Architectural and detailed design to address system requirements.
- c. **Coding Standards:** Writing consistent, readable, and reusable code following established standards.
- d. **Testing:** Systematic testing at various stages – unit, integration, system, and acceptance testing.
- e. **Documentation:** Maintaining clear and comprehensive documentation for future maintenance and upgrades.
- f. **Project Management:** Tracking progress, managing resources, and mitigating risks throughout the software lifecycle.

1.4. Software Myths

There are many myths surrounding software development, often leading to miscommunication and project failures.

Management Myths:

- 'We have the best tools, so we don't need a defined process.'
- 'Adding more developers will solve our deadline issues'.

Customer Myths:

- 'Changing requirements is easy and inexpensive.'
- 'Only the final working program matters.'
- A simple objective is enough to begin development.

Practitioner Myths:

- Once a program works, the job is done.
- 'Documentation is unnecessary because the code explains itself.'

These myths need to be addressed and corrected to ensure a realistic and efficient development process.

2. PROCESS MODELS

A **Software Process Framework** is a structured approach that defines the steps, tasks, and activities involved in software development. This framework serves as a **foundation for software engineering**, guiding the development team through various stages to ensure a **systematic and efficient process**. A Software Process Framework helps in project planning, risk management, and quality assurance by detailing the chronological order of actions.

Advantages of Software Process Framework

A **Software Process Framework** offers numerous benefits that streamline the **software development process** and enhance the quality and efficiency of the final product. Here are some key advantages:

- a. **Increased Productivity:** Frameworks provide pre-built components and tools, allowing developers to focus on specific application logic rather than reinventing the wheel.
- b. **Consistent Quality:** By following best practices and standardized processes, frameworks help ensure consistent code quality and structure across the project.
- c. **Reduced Development Time:** With ready-to-use templates and libraries, developers can significantly cut down on the time needed to build applications from scratch.
- d. **Better Maintainability:** A structured framework makes the code base more organized and easier to understand, which simplifies maintenance and updates.
- e. **Enhanced Security:** Frameworks often include built-in security features and follow industry best practices, reducing the risk of vulnerabilities.
- f. **Scalability:** Frameworks are designed to handle growth, making it easier to scale applications as user demand increases.

Dis-advantages of Software Process Framework

While **Software Process Framework** offer several advantages, they also come with certain drawbacks that developers and organizations should consider:

- a. **Learning Curve:** Frameworks often have a steep learning curve, requiring developers to invest time and effort in understanding the framework's architecture, conventions, and best practices.
- b. **Restrictions:** Some frameworks impose constraints and limitations on how developers can design and implement certain features, potentially limiting flexibility and creativity.
- c. **Complexity Overhead:** In some cases, frameworks introduce unnecessary complexity, especially for smaller or simpler projects, which can lead to over-engineering.

- d. **Performance Overhead:** Using a framework may introduce additional layers of abstraction and overhead, which can impact the performance of the application, particularly in resource-intensive environments.
- e. **Vendor Lock-in:** Depending heavily on a specific framework can lead to vendor lock-in, making it challenging to switch to alternative technologies or frameworks in the future.

2.1. Generic Software Process Model

The **Generic Process Model** represents a high-level, abstract framework for developing software. It provides a **generalized set of activities** that can be adapted to most software projects, regardless of size, complexity, or development methodology (e.g., waterfall, agile). The model ensures that software development is organized, efficient, and delivers quality results.

The five **framework activities** are:



1. Communication

Purpose: Understand what the customer needs.

▮ **Description:**

This is the **requirements gathering** stage. Developers interact with stakeholders (customers, users, managers) to collect the functional and non-functional requirements of the

software.

Clear communication here reduces misunderstandings later.

▮ **Typical activities:**

- ▮ Requirement elicitation through interviews, questionnaires, or observation
- ▮ Identifying stakeholders and their expectations
- ▮ Documenting and validating requirements
- ▮ Creating **Software Requirement Specification (SRS)**
- ▮ **Output:** A **clear, validated requirement document** that becomes the foundation for planning.

Example:

For an online banking application, the communication phase would identify features like account balance checking, fund transfers, and security requirements.

2. Planning

Planning means thinking before doing.

Purpose: Create a roadmap for software development.

▮ **Description:**

This step defines the **project plan**, estimates resources, schedules tasks, and identifies risks. Planning ensures the team knows *what*, *when*, and *how* the work will be done.

▮ **Typical activities:**

- ▮ Estimating effort, time, and cost
- ▮ Resource allocation (team members, hardware, software tools)
- ▮ Risk analysis and mitigation strategies
- ▮ Preparing a detailed **project schedule** (e.g., Gantt chart)

▮ **Output:** A **Project Plan Document (PPD)** with schedules, milestones, and resource allocation.

▮ **Example:**

For the banking app, planning may involve allocating 2 developers for the login module, 3 for transaction modules, with a timeline of 4 months.

3. Modeling

Purpose: Convert requirements into a blueprint for development.

▮ **Description:**

This phase involves **designing** both the system's architecture and its detailed components. It bridges the gap between "what the customer wants" and "how to implement it."

▮ **Typical activities:**

- ▮ Creating **data models** (ER diagrams)
- ▮ Designing **architectural models** (high-level structure)
- ▮ Designing **user interface (UI)** layouts
- ▮ Defining algorithms and workflows

▮ **Output:**

Design documents such as:

- ▮ **High-Level Design (HLD)**
- ▮ **Low-Level Design (LLD)**
- ▮ UI mock-ups

▮ **Example:**

For the banking app, modelling would define the database structure for accounts, the architecture for client-server communication, and wireframes for the user dashboard.

4. Construction

Purpose: Build and verify the actual software.

▮ **Description:**

The software is **coded** based on the design documents and then **tested** to ensure it meets requirements.

▮ **Typical activities:**

- ▮ Writing source code in the chosen programming language
- ▮ Unit testing (checking individual components)
- ▮ Integration testing (verifying module interaction)
- ▮ Debugging and fixing errors

▮ **Output:** A **tested, working software product** ready for deployment.

▮ **Example:**

Developers write the banking app code in Java or Python, test the login system, transaction processing, and validate them against the requirements.

5. Deployment

Purpose: Deliver the software to the end-users and provide ongoing support.

▮ **Description:**

This stage includes **installing, configuring**, and making the system operational in the customer environment.

It also involves **maintenance** to fix bugs, update features, and improve performance.

▮ **Typical activities:**

- ▮ Deployment to production servers or app stores
- ▮ User training and documentation
- ▮ Post-deployment support
- ▮ Corrective, adaptive, and perfective maintenance

▮ **Output:** Fully functional software available to the customer.

▮ **Example:**

The banking app is deployed on the bank's server, staff are trained, and future updates (e.g., biometric login) are planned.

Activity	Purpose	Key Outputs
Communication	Gather and understand needs	Requirements document (SRS)
Planning	Roadmap for development	Project Plan Document (PPD)
Modelling	Design system architecture	HLD, LLD, UI mock-ups
Construction	Build and test the software	Tested working software
Deployment	Deliver and maintain product	Operational system, maintenance updates

2.2. Process Assessment and Improvement

In software engineering, **Process Assessment** and **Process Improvement** are activities aimed at **evaluating, understanding, and enhancing** the way software is developed.

The goal is to ensure that processes are **efficient, predictable, and capable of delivering high-quality software** within time and budget constraints.

1. Process Assessment

Definition:

A structured evaluation of an organization's software development processes to determine their **current state of maturity, capability, and effectiveness.**

Purpose:

- ▮ Identify strengths and weaknesses in current processes
- ▮ Compare processes against industry standards and best practices
- ▮ Provide a baseline for improvement efforts

Key Steps in Process Assessment:

1. Define Assessment Scope

- ▮ Decide which projects, teams, or departments to evaluate.

2. Select Assessment Model

- ▮ Commonly used models include:
 - ▮ **CMMI (Capability Maturity Model Integration)**
 - ▮ **ISO/IEC 15504 (SPICE)**
 - ▮ **ISO 9001** for quality management

3. Data Collection

- ▮ Interviews with team members
- ▮ Reviewing documents, project reports, and metrics
- ▮ Observing work practices

4. Process Evaluation

- ▮ Compare actual practices with model requirements

5. Reporting

- ▮ Prepare an assessment report showing process strengths, weaknesses, and improvement areas.

Outputs:

- ▮ Assessment report with maturity/capability rating
- ▮ Recommendations for process changes

2. Process Improvement

Definition:

A systematic effort to enhance the **effectiveness, efficiency, and adaptability** of software processes, based on assessment results.

Goals:

- Reduce defects and rework
- Improve productivity and predictability
- Increase customer satisfaction
- Lower development costs

Principles of Process Improvement:

1. **Understand Current Process** (from assessment)
2. **Set Measurable Goals** (e.g., reduce defect rate by 20%)
3. **Involve Stakeholders** (management, developers, testers)
4. **Use a Model for Guidance** (e.g., CMMI or ISO)
5. **Measure and Monitor Progress** (process metrics)

3. Common Process Improvement Models

- **CMMI**: Focuses on five maturity levels, from Initial (ad-hoc) to Optimizing (continuous improvement).
- **ISO/IEC 15504 (SPICE)**: Rates processes on a capability scale (Level 0 to Level 5).
- **Six Sigma**: Reduces process variation and defects using statistical methods.
- **Lean Software Development**: Eliminates waste and improves efficiency.
- **Agile Process Improvement**: Uses iterative feedback loops and retrospectives.

4. Process Improvement Cycle

Often follows the **PDCA (Plan-Do-Check-Act)** or **DMAIC** (Define-Measure-Analyze-Improve-Control) cycle.

1. **Plan** – Identify areas for improvement and set goals
2. **Do** – Implement changes in a controlled environment
3. **Check** – Measure results against goals
4. **Act** – Standardize successful changes and repeat the cycle

5. Benefits of Process Assessment & Improvement

- Higher quality software
- Predictable delivery times
- Lower development costs
- Better team coordination
- Continuous learning and adaptation

6. Example

A company performs a **CMMI-based assessment** and finds:

- Lack of formal requirement management
- Inconsistent testing practices

Improvements Implemented:

- Introduced a formal SRS (Software Requirement Specification) process
- Standardized test case documentation and review
- Introduced automated testing tools

Result:

- 25% reduction in defects after release
- 15% faster delivery times

Process assessment and improvement involves evaluating the current software process and implementing changes for better quality and efficiency.

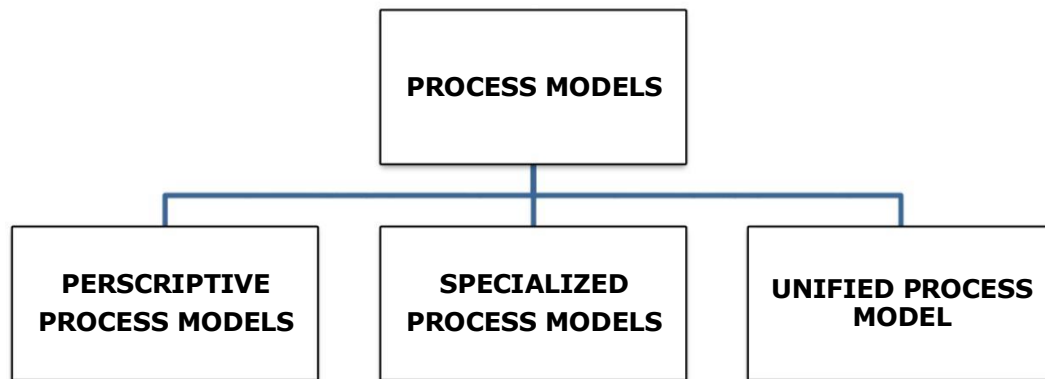
Key models include:

- CMMI (Capability Maturity Model Integration)
- ISO/IEC 15504 (SPICE)

These models help organizations to achieve process maturity and quality assurance.

6. Process Models

A process model for software engineering is chosen based on the nature of the project and application, the methods and tools to be used, and the controls and deliverables that are required. Process Models can be broadly classified as follows:-

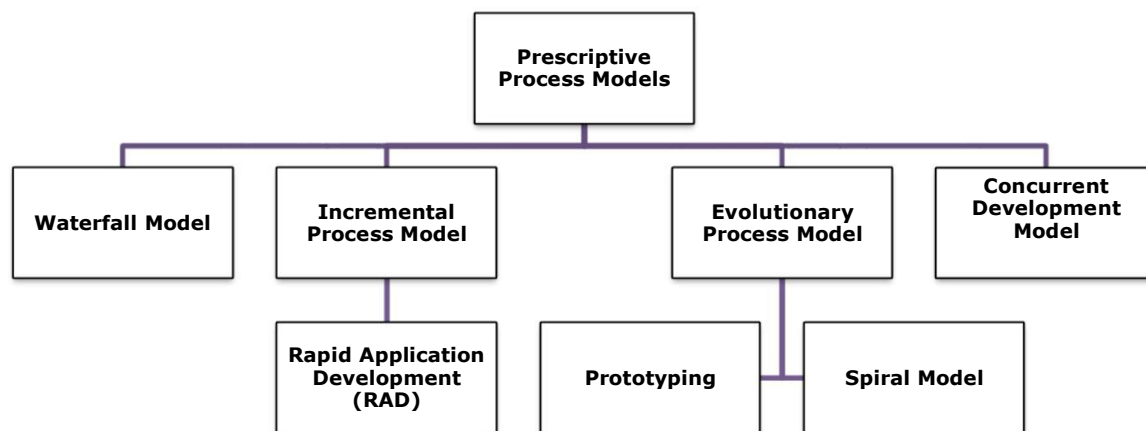


6.1. *PERSCRIPTIVE PROCESS MODELS*

Prescriptive Process Models defines a distinct set of activities, actions, tasks, milestones, and work products that are required to engineer high-quality software. Prescriptive Process Models are traditional process models and they provide structure to software engineering work and provide an effective road map for software teams. The dominant issues in these models are order and project consistency. These models are collectively named “Prescriptive Process Models” because they prescribe a set of process elements for each project, some of the process elements are:-

- a. Framework activities
- b. Software engineering actions
- c. Tasks
- d. Work Products
- e. Quality Assurance
- f. Change Control Mechanism

Each process model prescribes a process flow.



In the following sections some of the prescriptive process models are described namely:-

- ❑ Waterfall Model
- ❑ Incremental Process Model
- ❑ Evolutionary Process Model
- ❑ Concurrent Development Model

Prescriptive process models provide structured guidelines for software development.

1. Waterfall Model

Waterfall Model also called classic life cycle or the linear sequential model suggests a systematic, sequential approach to software development. It's a systematic, sequential, approach to software development that begins with customer communication, and moves forward with planning, modeling, construction and ends with deployment as shown in Figure 1. This is the oldest software lifecycle model. The main stages in this processing models are:

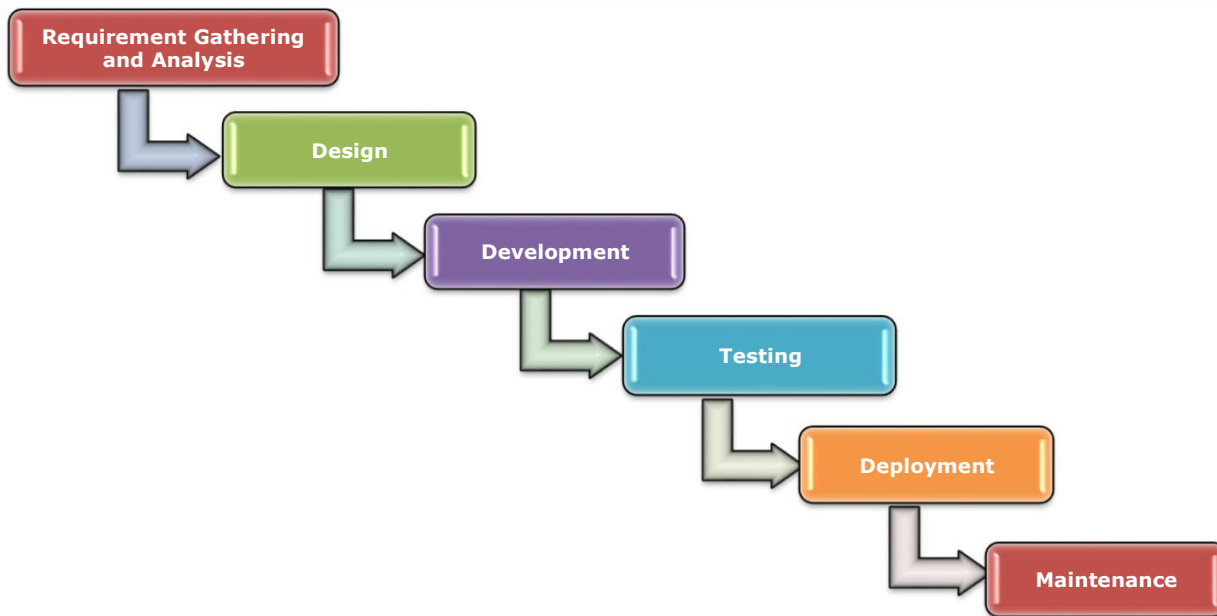


Figure 1 Waterfall Model

I. Requirement gathering and Analysis

This phase, aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

- a. **Requirement Gathering and Analysis:** Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).
- b. **Requirement Specification:** These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

II. Design

The goal of this **Software Design Phase** is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A **Software Design Document** is used to document all of this effort (SDD).

- a. **High-Level Design (HLD):** This phase focuses on outlining the broad structure of the system. It highlights the key components and how they interact with each other, giving a clear overview of the system's architecture.
- b. **Low-Level Design (LLD):** Once the high-level design is in place, this phase zooms into the details. It breaks down each component into smaller parts and provides specifics about how each part will function, guiding the actual coding process.

III. Development

In the **Development Phase** software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not. In this phase, developers begin writing the actual source code based on the designs created earlier, the goal is to transform the design into working code using the most suitable programming languages. Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

IV. Testing

Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

Alpha testing: Alpha testing is the system testing performed by the development team.

Beta testing: Beta testing is the system testing performed by a friendly set of customers.

Acceptance testing: After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

V. Deployment

Once the software has been thoroughly tested, it's time to deploy it to the customer or end-users. This means making the software ready and available for use, often by moving it to a live or staging environment.

During this phase, we also focus on helping users get comfortable with the software by providing training, setting up necessary environments, and ensuring everything is running smoothly. The goal is to make sure the system works as expected in real-world conditions and that users can start using it without any hitches.

VI. Maintenance

In **Maintenance Phase** is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.

1. **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
2. **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
3. **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

Features of Waterfall Model

Following are the features of the waterfall model:

▮ Sequential Approach:

The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.

▮ Document-Driven

The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.

▮ Quality Control

The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.

▮ Rigorous Planning

The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Overall, the waterfall model is used in situations where there is a need for a highly structured and systematic approach to software development. It can be effective in ensuring that large, complex projects are completed on time and within budget, with a high level of quality and customer satisfaction.

Importance of Waterfall Model

Following are the importance of waterfall model:

- a. **Clarity and Simplicity:** The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.
- b. **Clearly Defined Phases:** The Waterfall Model phases each have unique inputs and outputs, guaranteeing a planned development with obvious checkpoints.
- c. **Documentation:** A focus on thorough documentation helps with software comprehension, maintenance, and future growth.
- d. **Stability in Requirements:** Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.
- e. **Resource Optimization:** It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.
- f. **Relevance for Small Projects:** Economical for modest projects with simple specifications and minimal complexity.

Example of Waterfall Model

Here we can take a Real world example of the Waterfall Model.

Real-Life Example of Waterfall Model: Developing an Online Banking System

1. Analysis

This phase will be tasked with gathering all the information available on customer banking requirements, transactions, security protocols, and devising the different parameters that'll be used for determining the core functionalities of the online banking system, such as account management, fund transfers, bill payments, and loan applications.

2. Design

In this example of the Waterfall Model, the design phase is all about fine-tuning the parameters established in the analysis phase. The system's architecture will be designed to manage sensitive data securely, avoid transactional errors, and ensure high performance. This includes database structure, user interface design, encryption protocols, and multi-factor authentication to protect user accounts.

3. Implementation

This all-important phase involves doing dummy runs of the online banking system with a provisional set of banking transactions and customer data to see the accuracy with which the system can handle transactions, balance inquiries, fund transfers, and bill payments. These results should be matched with results from banking experts and auditors who ensure compliance with banking regulations and accuracy in transactions.

4. Testing

As with any example of the Waterfall Model, the testing phase is about ensuring that all features of the online banking system function smoothly. This includes testing for security vulnerabilities, transaction accuracy, performance under heavy load, and user interface responsiveness. Special attention is given to testing secure logins, data encryption, and ensuring that sensitive data is handled correctly throughout the system.

5. Maintenance

In the final phase, the online banking system should be checked for any necessary updates or alterations that may be required, besides the expected inclusion of new features or changes in banking regulations. Regular updates will also be needed for security patches, performance improvements, and the addition of new services like mobile banking, instant loans, or personalized financial advice.

Advantages of Waterfall Model

The classical waterfall model is an idealistic model for software development. It is very simple, so it can be considered the basis for other software development life cycle models. Below are some of the major advantages of this SDLC model.

- a. Easy to Understand:** The Classical Waterfall Model is very simple and easy to understand.
- b. Individual Processing:** Phases in the Classical Waterfall model are processed one at a time.
- c. Properly Defined:** In the classical waterfall model, each stage in the model is clearly defined.
- d. Clear Milestones:** The classical Waterfall model has very clear and well-understood milestones.
- e. Properly Documented:** Processes, actions, and results are very well documented.
- f. Reinforces Good Habits:** The Classical Waterfall Model reinforces good habits like define-before-design and design-before-code.
- g. Working:** Classical Waterfall Model works well for smaller projects and projects where requirements are well understood.

Disadvantages of Waterfall Model

The Classical Waterfall Model suffers from various shortcomings we can't use it in real projects, but we use other software development lifecycle models which are based on the classical waterfall model. Below are some major drawbacks of this model.

- a. **No Feedback Path:** In the classical waterfall model evolution of software from one phase to another phase is like a waterfall. It assumes that no error is ever committed by developers during any phase. Therefore, it does not incorporate any mechanism for error correction.
- b. **Difficult to accommodate Change Requests:** This model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but the customer's requirements keep on changing with time. It is difficult to accommodate any change requests after the requirements specification phase is complete.
- c. **No Overlapping of Phases:** This model recommends that a new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained. To increase efficiency and reduce cost, phases may overlap.
- d. **Limited Flexibility:** The Waterfall Model is a rigid and linear approach to software development, which means that it is not well-suited for projects with changing or uncertain requirements. Once a phase has been completed, it is difficult to make changes or go back to a previous phase.
- e. **Limited Stakeholder Involvement:** The Waterfall Model is a structured and sequential approach, which means that stakeholders are typically involved in the early phases of the project (requirements gathering and analysis) but may not be involved in the later phases (implementation, testing, and deployment).
- f. **Late Defect Detection:** In the Waterfall Model, testing is typically done toward the end of the development process. This means that defects may not be discovered until late in the development process, which can be expensive and time-consuming to fix.
- g. **Lengthy Development Cycle:** The Waterfall Model can result in a lengthy development cycle, as each phase must be completed before moving on to the next. This can result in delays and increased costs if requirements change or new issues arise.

When to Use Waterfall Model?

Here are some cases where the use of the Waterfall Model is best suited:

- 1. **Well-understood Requirements:** Before beginning development, there are precise, reliable, and thoroughly documented requirements available.

2. **Very Little Changes Expected:** During development, very little adjustments or expansions to the project's scope are anticipated.
3. **Small to Medium-Sized Projects:** Ideal for more manageable projects with a clear development path and little complexity.
4. **Predictable:** Projects that are predictable, low-risk, and able to be addressed early in the development life cycle are those that have known, controllable risks.
5. **Regulatory Compliance is Critical:** Circumstances in which paperwork is of utmost importance and stringent regulatory compliance is required.
6. **Client Prefers a Linear and Sequential Approach:** This situation describes the client's preference for a linear and sequential approach to project development.
7. **Limited Resources:** Projects with limited resources can benefit from a set-up strategy, which enables targeted resource allocation.

The Waterfall approach involves less user interaction in the product development process. The product can only be shown to end user when it is ready.

Applications of Waterfall Model

Here are some application of SDLC waterfall model:

- a. **Large-scale Software Development Projects:** The Waterfall Model is often used for large-scale software development projects, where a structured and sequential approach is necessary to ensure that the project is completed on time and within budget.
- b. **Safety-Critical Systems:** The Waterfall Model is often used in the development of safety-critical systems, such as aerospace or medical systems, where the consequences of errors or defects can be severe.
- c. **Government and Defense Projects:** The Waterfall Model is also commonly used in government and defense projects, where a rigorous and structured approach is necessary to ensure that the project meets all requirements and is delivered on time.
- d. **Projects with well-defined Requirements:** The Waterfall Model is best suited for projects with well-defined requirements, as the sequential nature of the model requires a clear understanding of the project objectives and scope.
- e. **Projects with Stable Requirements:** The Waterfall Model is also well-suited for projects with stable requirements, as the linear nature of the model does not allow for changes to be made once a phase has been completed.

6.2 Incremental Process Model

Incremental Process Model combines elements of the waterfall model applied in an iterative fashion. Each linear sequence produces deliverable “increments” of the software as shown in Figure 11.2. The first increment is often a core product.

In core product:

- Basic requirements are addressed.
- Many supplementary features remain undelivered.

The core product is used by the customer (or undergoes detailed evaluation). Based on evaluation results, a plan is developed for the next increment. Plan addresses modification of core product, Delivery of additional features and functionality. This kind of development of plan is repeated after every increment. Earlier increments are stripped down versions of final product. Final product is the software developments after a series of increments.

Incremental Process Model is applied under following conditions:-

- I. When we have to provide a limited set of functionality to users quickly and we need to refine and expand on that functionality in later software releases.
- II. When sufficient staffing is unavailable for complete implementation of software.

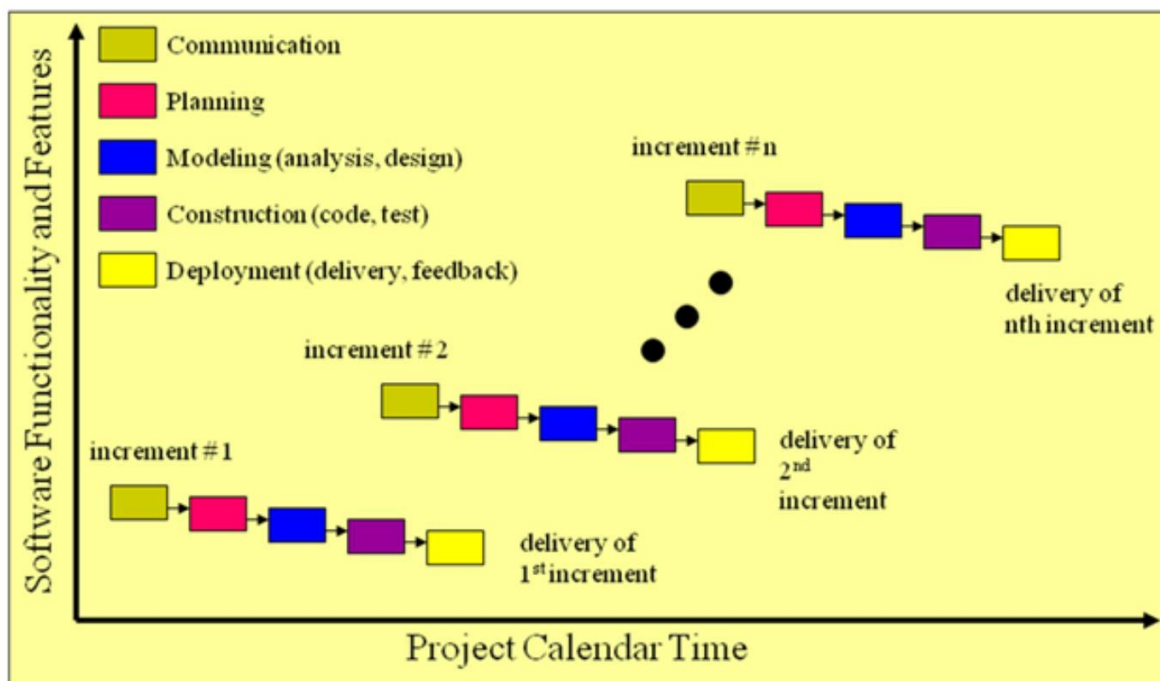


Fig 11. 2: The incremental model

6.3 *Evolutionary Process Models*

Evolutionary models are iterative in nature. Iterative nature of this model enables us to develop increasingly more complete version of the software. Two types are introduced, namely *Prototyping* and *Spiral*.

1. **Prototyping Model**

Prototyping model is used when Customer defines a set of general objectives but does not identify detailed requirements for functions and features or Developer may be unsure of the efficiency of an algorithm, the form that human computer interaction should take. The steps involved in prototyping model are,

As shown in Figure 11.3(i) Prototyping begins with communication by meeting with stakeholders to define the objective, identify whatever requirements are known, and outline areas where further definition is mandatory. A quick plan for prototyping and modeling (quick design) occur. Quick design focuses on a representation of those aspects the software that will be visible to end users (interface and output). Design leads to the construction of a prototype which will be deployed and evaluated. Stakeholder's comments will be used to refine requirements.

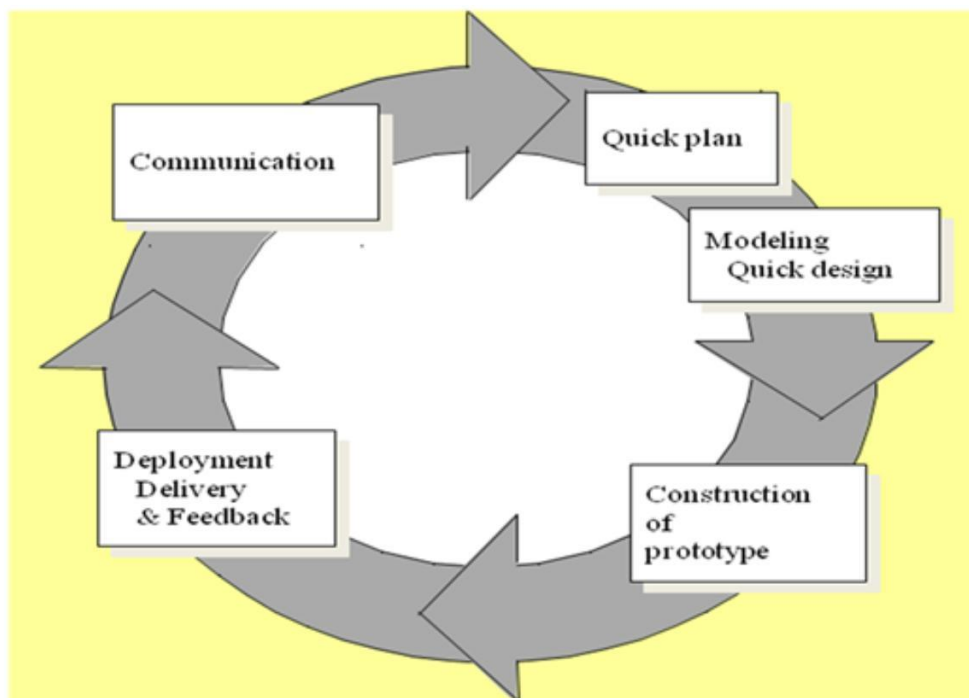


Figure: Prototyping Model

- || Both stakeholders and software engineers like the prototyping paradigm.
- || Users get a feel for the actual system, and developers get to build something immediately.

Advantages of Prototyping Model

- a. The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- b. New requirements can be easily accommodated as there is scope for refinement.
- c. Missing functionalities can be easily figured out.
- d. Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- e. The developed prototype can be reused by the developer for more complicated projects in the future.
- f. Flexibility in design.
- g. Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.
- h. Prototyping can be used to test and validate design decisions, allowing for adjustments to be made before significant resources are invested in development.
- i. Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.
- j. Prototyping can facilitate communication and collaboration among team members and stakeholders, improving overall project efficiency and effectiveness.
- k. Prototyping can help bridge the gap between technical and non-technical stakeholders by providing a tangible representation of the product.

Disadvantages of the Prototyping Model

- I. Costly concerning time as well as money.
- II. There may be too much variation in requirements each time the prototype is evaluated by the customer.
- III. Poor Documentation due to continuously changing customer requirements.
- IV. It is very difficult for developers to accommodate all the changes demanded by the customer.
- V. There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- VI. After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- VII. Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- VIII. The customer might lose interest in the product if he/she is not satisfied with the initial prototype.

Applications of Prototyping Model

- a. The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- b. The prototyping model can also be used if requirements are changing quickly.
- c. This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- d. The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.

e. Examples:

I. Web Development:

Prototypes can be used to create a working model of a website to identify potential issues and gather feedback on the user experience.

II. Game Development:

Prototypes, often called "vertical slices" or "proofs of concept," help test core gameplay mechanics and identify potential problems early in the development cycle.

III. User Interface (UI) Design:

UI designers use prototypes to explore different design concepts, test user interactions, and iterate on the look and feel of an application's interface.

IV. Software Development:

Prototypes are used to validate product requirements, test different functionalities, and ensure that the final product meets user needs. This can be done using rapid prototyping or evolutionary prototyping approaches.

V. Product Development:

Prototypes can range from simple sketches and mockups to functional models, helping to evaluate product ideas and gather feedback from stakeholders.

VI. Mobile App Development:

Prototyping tools like **Mockplus**, **Sketch**, **Figma**, and **Adobe XD** allow for the creation of interactive prototypes to test user flows and interactions in mobile applications.

VII. Startup Development:

Prototyping is crucial for startups to quickly test their ideas, gather user feedback, and iterate on their product before investing heavily in full-scale development.

2. Spiral Model

The Spiral Model is a well-known and widely used software development model. It has a unique approach to handling the complexities of software development, which sets it different from other models like Waterfall or Agile. It encapsulates the essence of other software development models while providing a flexible framework for iterative and risk-driven project management. This makes the spiral model referred to as a "meta model." A Meta model is a superior, higher-level model that has the structure and principles of other software development life cycle models. In general, a Meta model serves as a model of models, facilitating a comprehensive and unified perspective on a subject or system.

Each phase is depicted as a spiral, signifying the iterative and cyclical nature of the model. Spiral Model couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model as shown in Figure. Spiral Model was proposed by Barry Boehm. It provides the potential for rapid development of increasingly more complete versions of the software. It is a *risk-driven process model* generator.

It has two main distinguishing features

- i. **Cyclic approach-** Incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.
- ii. **A set of *anchor point milestones*-** A combination of work products and conditions that are attained along the path of the spiral.

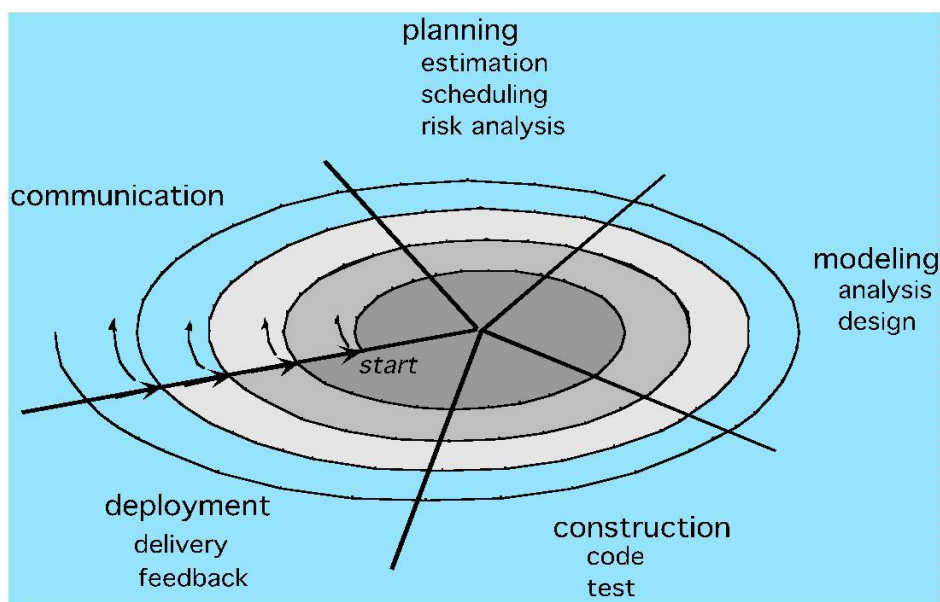


Figure: Spiral Model

A series of evolutionary releases are delivered. During the early iterations, the release might be a model or prototype. During later iterations, increasingly more complete version of the engineered system is produced. The first circuit in the clockwise direction might result in the product specification; subsequent passes around the spiral might be used to develop a prototype and then progressively more sophisticated versions of the software. Each pass results in adjustments to the project plan. Cost and schedule are adjusted based on feedback. Also, the number of iterations will be adjusted by project manager. Unlike other process models that end when software is delivered, the spiral model can be adapted to apply throughout the life of the computer s/w.

The circuits around the spiral might represent

- || Concept development project
- || New Product development project
- || Product enhancement project

The spiral model demands a direct consideration of technical risks at all stages of the project. Each of the circuit consists of following activities:-

- 1. Communication**
- 2. Planning**
- 3. Modeling**
- 4. Construction**
- 5. Deployment**

1. Communication

It involves communication among the customer and other stake holders; encompasses requirements gathering. The purpose of communication is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

2. Planning

Planning activity creates a “map” that helps the software team as it makes journey. This plan is called “software project plan”. Software project plan defines the software engineering work by describing the technical task to be conducted, the risks involved, resources that will be required, work products to be produced, and a work schedule.

3. Modeling (Analyze, Design)

It encompasses the creation of models to better understand the requirements and the design.

4. Construction (Code, Test)

Combines code generation and testing to uncover errors.

5. Deployment

Involves delivery of software to the customer.

Customer evaluates the delivered product and provides feedback based on evaluation.

Benefits of using Spiral Model:-

1. The spiral model is a realistic approach to the development of large-scale systems and software.
2. Because software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level.
3. The spiral model uses prototyping as a risk reduction mechanism but, more important, enables the developer to apply the prototyping approach at any stage in the evolution of the product.
4. It maintains the systematic stepwise approach suggested by the classic life cycle but incorporates it into an iterative framework that more realistically reflects the real world.
5. Direct consideration of technical risks at all stages of the project and, if properly applied, should reduce risks before they become problematic.

Dangers of using Spiral Model:-

1. It may be difficult to convince customers (particularly in contract situations) that the evolutionary approach is controllable.
2. It demands considerable risk assessment expertise and relies on this expertise for success.
3. If a major risk is not uncovered and managed, problems will undoubtedly occur.
4. Finally, the model has not been used as widely as the linear sequential or prototyping paradigms. It will take a number of years before efficacy of this important paradigm can be determined with absolute certainty.

Applications of Spiral Models

The Spiral Model is well-suited for large, complex, and high-risk projects where requirements are likely to change, and customer feedback is crucial. Examples include: developing operating systems like **Microsoft Windows**, game development, and creating complex systems like **Intelligent Transportation Systems**. It's also used in **e-commerce platforms**, electronic health record systems, and **mobile application** development.

6.4 *Concurrent Development Model*

In Concurrent Development Model each of the activities involved in software engineering process are represented using different states. The concurrent development model is sometimes called concurrent engineering. All activities exist concurrently but reside in different states. It defines a series of events that will trigger transitions from state to states. This one is applicable to all types of s/w development. It defines a network of activities. Events generated at one point in the process network trigger transitions among the states.

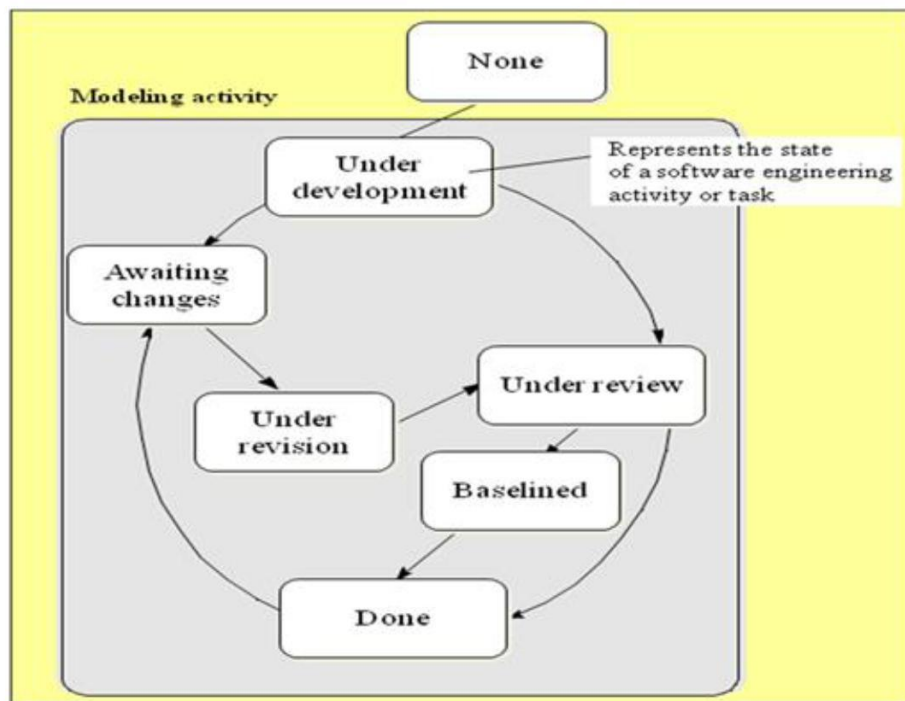


Figure: One element of the concurrent process model

The figure provides a schematic representation of one software engineering activity (modeling) using a concurrent modeling approach. The activity modeling may be in one of the states. For example, early in a project the customer communication activity has completed its first iteration and exists in the awaiting changes state. If, however, the customer indicates that changes in requirements must be made, the analysis activity moves from the under development state into the awaiting changes state.

6.5 *Agile Model*

Agile Development Model is a combination of iterative and incremental models, that is, it is made up of iterative and incremental models. In Agile model, focus is given to process adaptability and customer satisfaction. In earlier times, iterative waterfall model was used to create software. But in today's time developers have to face many problems. The biggest problem is that in the middle of software development, the customer asks to make changes in the software. It takes a lot of time and money to make these changes. So to overcome all these shortcomings, the agile model was proposed in the 1990s.

The Agile Model was created mainly to make changes in the middle of software development so that the software project can be completed quickly.

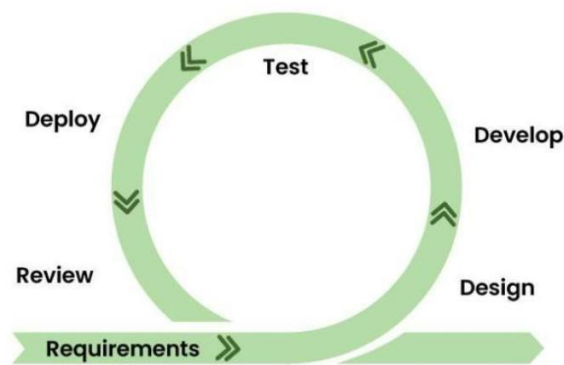


Figure: Agile-model

In the agile model, the software product is divided into small incremental parts. In this, the smallest part is developed first and then the larger one and each incremental part is developed over iteration. Each iteration is kept small so that it can be easily managed. And it can be completed in two-three weeks. Only one iteration is planned, developed and deployed at a time.

6.6 *Specialized Process Models*

Specialized Process Models take on many of the characteristics of one or more of the conventional models. These models are tend to be applied when a narrowly defined software engineering approach is chosen. There are three kinds of Specialized Process Models are used:-

- i. Component-Based Development
- ii. The Formal Methods Model
- iii. Aspect-Oriented Software Development

1. Component-Based Development

- || Commercial off-the-shelf (COTS) software components can be used.
- || Components should have well-defined interfaces.
- || Incorporates many of the characteristics of the spiral model.
- || Evolutionary in nature.
- || Candidate components should be identified first.
- || Components can be designed as either conventional software modules or object-oriented classes or packages of classes

2. The Formal Methods Model

- || Encompasses a set of activities that leads to formal mathematical specification of computer software
- || Have provision to apply a rigorous, mathematical notation.
- || Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily – not through ad hoc review, but through the application of mathematical analysis
- || Offers the promise of defect-free software
- || Cleanroom Software Engineering is a variation of the Formal Methods Model.
- || **Drawbacks of Formal Methods Model:-**
 - The development of formal models is currently quite time-consuming and expensive.
 - Extensive training is required.
 - It is difficult to use the models as a communication mechanism for technically unsophisticated customers.

3. Aspect-Oriented Software Development

- || Aspect Oriented software development (AOSD)—provides a process and methodological approach for defining, specifying, designing, and constructing aspects.
- || It is likely that AOSD will adopt characteristics of both the spiral and concurrent process models.

8. Unified Process Model

Unified Process Model is a “use-case driven, architecture-centric, iterative and incremental” software process closely aligned with the Unified Modeling Language (UML) to model and develop object-oriented system iteratively and incrementally. UP is an attempt to draw on the best features and characteristics of conventional s/w process models and also implements many of the best principles of agile software development. UP is a framework for object-oriented software engineering using UML (Unified Modeling Language).

Figure (i) shows different Phases of Unified Process:-

1. Inception
2. Elaboration
3. Construction
4. Transition
5. Production

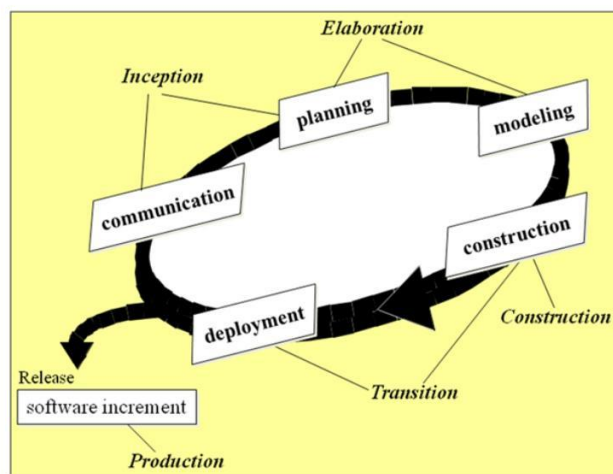


Figure (i) Phases of Unified Process

1. Inception

- ┃ Encompasses both customer communication and planning activities.
- ┃ Fundamental business requirements are described through a set of preliminary use-cases
- ┃ A use-case describes a sequence of actions that are performed by an actor (Eg. a person, a machine, another system) as the actor interacts with the software
- ┃ A rough architecture for the system is also proposed.

2. Elaboration

- ┃ Encompasses customer communication and modeling activities.
- ┃ Refines and expands the preliminary use-cases
- ┃ Expands the architectural representation to include five different views of the software
- ┃ The use-case model
- ┃ The analysis model
- ┃ The design model
- ┃ The implementation model
- ┃ The deployment model
- ┃ In some cases, elaboration creates an “executable architectural baseline” that represents a “first cut” executable system.

3. Construction

- ┃ Makes each use-case operational for end-users
- ┃ As components are being implemented, unit tests are designed and executed for each
- ┃ Integration activities (component assembly and integration testing) are conducted
- ┃ Use-cases are used to derive a suite of acceptance tests

4. Transition

- ┃ Software is given to end-users for beta testing
- ┃ The software team creates the necessary support information such as
- ┃ User manuals
- ┃ Trouble-shooting guides
- ┃ Installation procedures
- ┃ At the conclusion of the transition phase, the software increment becomes a usable software release

5. Production

- ┃ Coincides with the deployment activity of the generic process
- ┃ The on-going use of the software is monitored
- ┃ Support for the operating environment (infrastructure) is provided
- ┃ Defect reports and requests for changes are submitted and evaluated.

Unified Process Work Products are:-**Inception**

- Vision document
- Initial use-case model

Elaboration

- Analysis model
- design model

Construction

- Implementation model,
- Deployment model
- Test model

Transition

- Delivered software
- Beta test reports
- General user feedback

Reference: Software Engineering: A Practitioner's Approach, Roger S. Pressman, Bruce R. Maxim, 9th Edition, McGraw Hill Education, 2024